Software Requirements Specification (SRS) Document

Room Booking Portal

Team-54

Members:

Harshit Singh Mehta (2024113008)

Chanakya Manikanta Reddy Arumalla (2024101120)

Mahanth Reddy (2024101017)

Haaris Mohammed Iqbal (2024101045)

Ishaan Shiv Kumar (2024101098)

Brief problem statement

Previously, room booking was managed through IMS, in which tracking room availability, scheduling, and handling conflicting requests was difficult. To streamline and optimize the booking process, we are developing a Room Booking Portal that integrates with IMS and utilizes its room data as the foundational source,thereby filling the previous gaps.

.The Room Booking Portal will automate this via CAS login, IMS(integrated management system)-integrated availability checks, CFS approvals, email notifications (including CDS for snacks), feedback forms, and role-based dashboards—using PHP/MySQL backend and HTML/Bootstrap/JS frontend—to enhance efficiency and resource management.

System requirements (Client side)

Hardware

- Any device capable of running a modern web browser (desktop, laptop, tablet, or smartphone)
- Minimum 2GB RAM
- Active network interface (wired or wireless)

Software

- Any modern web browser — Google Chrome (v90+), Mozilla Firefox (v88+), Microsoft Edge (v90+).
- JavaScript must be enabled in the browser
- No additional software installation required as the website will be rendered in the browser

Network

- Active connection to IIIT-H's campus network.
- VPN access required for off-campus use (subject to institute IT policy)

- Minimum recommended bandwidth: 2 Mbps

Authentication

- Valid IIIT-H CAS (Central Authentication Service) credentials
- Applicable only to registered Staff and Faculty members
- Students are not permitted direct access to the portal

Users profile

There will be mainly three different types of users of IIIT-H. They are:

· Faculty: They will be using the system in user mode. They have vast experience in terms of familiarity with computer systems and softwares so a complex software wouldn't be too hard on them.

· Staff: They also will be using the system in user mode. They also have a pretty good experience but preferably it would be better to create a simple UI keeping them in perspective.

· CFS staff: These are the users who are going to be using the system in admin mode. They will be able to approve and deny requests that have been sent by the users and also will be able to look at the reports generated as well as the statistics of room bookings on a timely basis.

Functional Requirements:-

<u>R1 – Core Functional Requirements</u>

FR-1. Role Based Login
A user authenticates via CAS to access the portal. The system validates credentials, creates a session based on role (staff/faculty or admin), and redirects the user to the appropriate dashboard.

FR-2. View Available Rooms
A logged-in user views a grid of rooms showing availability (green for available, red for unavailable) based on type, capacity, and selected date/time. The system fetches room data from IMS and checks for conflicts.

FR-3. Search Rooms
A user applies filters such as room type (classroom/meeting), capacity, or date to narrow down available rooms. The system updates the display in real time.

FR-4. Submit Booking Request
A user selects a room, specifies start/end time and purpose, and submits the request to CFS. The system saves the request as *pending* and sends a notification email to CFS.

FR-5. View Pending Requests
An admin views a list of all pending booking requests with details such as user, room, time, and purpose, sorted by submission time.

FR-6. Approve Booking
An admin reviews a pending request and approves it, updating the status in the database. The system sends a confirmation email to the user and marks the room as unavailable for that slot.

FR-7. Decline Booking
An admin rejects a pending request, adds notes for the reason, updates the status, and notifies the user via email.

FR-8. Logout
A user or admin ends their session securely. The system clears session data and redirects to the login page.

R2 – Additional Functional Requirements

FR-9. Cancel Booking
A user cancels an existing pending or approved booking before the start time. The system updates the status to *cancelled*, notifies CFS via email, and releases room availability.

FR-10. Snack / Tech Options (CDS Integration)
During booking submission, a user selects optional snacks or tech support. The system records these details, and upon approval, triggers notifications to CDS.

FR-11. Submit Feedback
After an event ends, a user receives a link and optionally submits feedback with rating and comments. The system stores the feedback and marks the booking as completed.

FR-12. Request More Details
An admin requests additional information for a pending booking by adding notes. The system emails the user and keeps the booking status as *pending*.

FR-13. View Statistics
An admin accesses charts and a calendar view of bookings filtered by time or status. The system visualizes booking data for resource management.

FR-14. View Booking History
A user views a list of their past and pending bookings, including room, time, and status, displayed in a tabular format.

Non-Functional Requirements (Properties)

Technical Constraints:

- Authentication shall be performed exclusively via CAS; password credentials should not be stored by the system.
- Room metadata must originate from IMS, not manually maintained.
- The system must reliably and securely transmit the approved snack data to the Campus Dining Services (CDS) system using standard, lightweight data formats (like JSON over a REST API).

Reliability:

- Invalid or expired CAS tickets shall not create a session.
- Booking submission shall be atomic—either fully stored as pending or not stored at all.
- The automated email notification to CFS must include a retry mechanism (e.g., up to 3 attempts) in case the mail server experiences temporary downtime.

Performance:

- Room availability computation (IMS fetch + conflict check) shall complete within acceptable time for interactive use.
- The logout process and redirection to the CAS login screen must complete in under 1 second.
- The system must update the database and release the room's availability in real-time (under 2 seconds) to prevent subsequent booking errors.
- The system must dispatch the clarification email to the user within 1 minute of the admin requesting clarification.
- The visual charts and calendars (using Chart.js and FullCalendar.js) must render within 3 seconds, even when querying large datasets spanning an entire academic year.
- The history table should utilize some level of optimization so that users with extensive booking histories experience page load times of under 2 seconds.

Scalability:

- Search functionality shall continue to perform acceptably as the number of rooms and bookings increases.
- Database queries generating these statistics must be properly indexed to ensure the admin dashboard does not suffer performance degradation as the booking history grows over time.

Process Constraint:

- Pending requests shall be ordered by submission time to ensure fair processing.

Consistency:

- Approval shall immediately block the room for the approved time slot across the system.

Concurrency:

- The system shall prevent race conditions where multiple admins attempt to approve conflicting requests.

Auditability:

- Decline actions shall store the rejection reason for future reference.

Security:

- Session tokens must be completely invalidated on both the client and server side immediately upon execution to prevent session hijacking

Usability:

- The UI for selecting optional items must clearly display any limits or constraints so users do not submit invalid requests.
- The feedback form must be fully responsive and easily navigable as users are likely to submit it on the go after an event.
- The tabular display must include immediate client-side sorting (e.g., by date or status) without requiring a full page refresh.

Data Integrity:

- The system must ensure that the original booking request data remains entirely intact and locked from other edits while its status is pending clarification from the user.

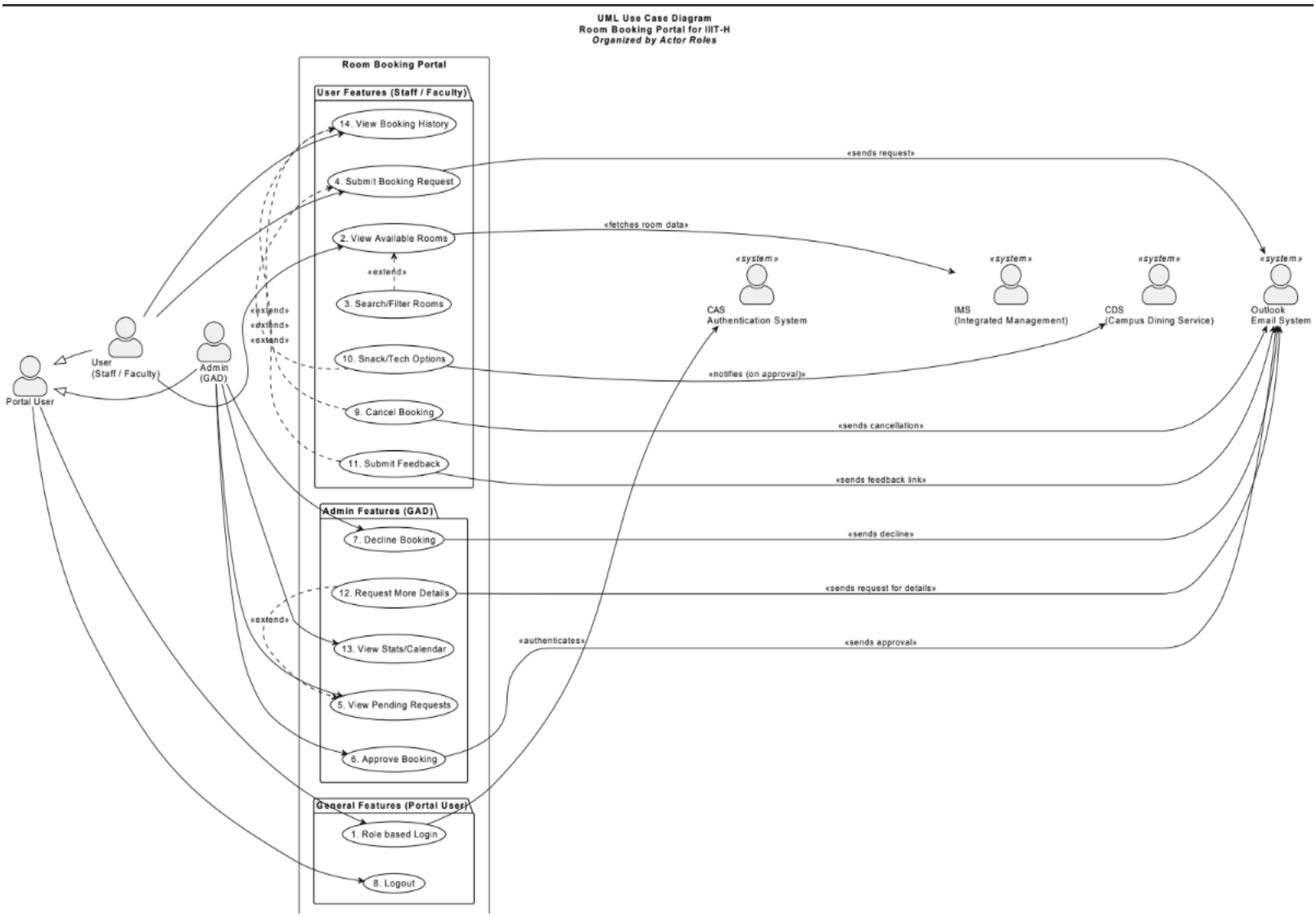
Feature requirements (described using use cases)

No.	Use Case Name	Description	Release
1.	Role based Login	**A user authenticates via CAS to access the portal. The system validates credentials and creates a session based on role (staff/faculty/admin as user). Upon success, the user is redirected to their dashboard.**	R1
2.	View Available Rooms	A logged-in user views a grid of rooms showing availability (green for available, red for unavailable) based on type, capacity, and selected date/time. The system fetches room data from IMS and checks for conflicts. This helps users quickly identify suitable options for booking.	R2
3.	Search/Filter Rooms	A user applies filters like room type (classroom/meeting), capacity, or date to narrow down available rooms. The system queries the database and updates the display in real-time. This ensures efficient selection without scrolling through all rooms.	R1
4.	Submit Booking Request	A user selects a room, specifies start/end time, purpose, and submits the request to CFS. The system saves it as 'pending' in the database and sends a notification email to CFS. This initiates the approval workflow.	R1
5.	View Pending Requests	An admin views a list of all pending booking requests with details like user, room, time, and purpose. The system fetches from the database and sorts by submission time. This enables efficient review and decision-making.	R2
6.	Approve booking	An admin reviews a pending request and approves it, updating the status in the database. The system sends a confirmation email to the user and marks the room as unavailable for that slot. This finalizes the booking for use.	R1
7.	Decline booking	An admin rejects a pending request, adding notes for the reason, and updates the status. The system notifies the user via email with the decline details.	R1
8.	Logout	A user or admin ends their session securely. The system clears the session data and redirects to the login page. This prevents unauthorized access after use.	R1
9.	Cancel Booking	A user cancels an existing pending or approved booking before the start time. The system updates the status to 'cancelled', notifies CFS via email, and releases the room availability. This provides flexibility for changes in plans.	R2
10.	Snack/Tech Options (CDS integration)	During booking submission, a user selects optional snacks . The system records these in the booking details for approval. If approved, it triggers notifications to CDS.	R2

11.	Submit Feedback	After an event ends, a user receives a link and submits feedback with rating and comments. The system stores it in the database and marks the booking as completed. This helps improve future services based on user input.	R2
12.	Request More Details	An admin requests additional information for a pending booking by adding notes. The system sends an email to the user prompting for updates and keeps the status as 'pending'. This allows clarification without outright rejection.	R1
13.	View stats/Calendar	An admin accesses charts and a calendar view of bookings, filtered by time or status. The system uses libraries like Chart.js and FullCalendar.js to visualize data from the database. This provides insights for resource management.	R2
14.	View booking history	A user accesses a list of their past and pending bookings, including details like room, time, and status. The system retrieves data from the bookings table and displays it in a table format. This allows users to track their requests easily.	R2

Use case diagram:

[(UML Diagram link)]



Use case description:

Use Case	UC-01
Use Case Name	Role based login
Overview:	**A user authenticates via CAS to access the portal. The system validates credentials and creates a session based on role (staff/faculty/admin as user). Upon success, the user is redirected to their dashboard.**
Actors:	Primary: User (Staff / Faculty) ,Supporting: CAS Authentication System
Properties:	● **Technical constraints:** Authentication shall be performed exclusively via CAS; password credentials should be not be stored by the system ● **Reliability:** Invalid or expired CAS tickets shall not create a session.
Pre condition:	The user is on the portal login page and has valid IIIT-H credentials. The CAS server is available.
Flow:	Main (success) Flow: 1. User clicks “Login with CAS”. 2. System redirects to CAS login page. 3. User enters IIIT-H username and password. 4. CAS validates credentials and returns success ticket. 5. System validates ticket, creates session with role ‘user’, and redirects to User Dashboard.
	Alternate Flows: 3a. Invalid credentials → CAS shows error → User remains on CAS page (loop to step 3).
Post Condition:	User is successfully logged in with an active session and is on the User Dashboard.

Use Case	UC-02
Use Case Name	View Available Rooms
Overview:	**A logged-in user views a grid of rooms showing availability (green for available, red for unavailable) based on type, capacity, and selected date/time. The system fetches room data from IMS and checks for conflicts. This helps users quickly identify suitable options for booking.**
Actors:	Primary: User (Staff / Faculty) Supporting: IMS System
Properties:	● **Performance:** Room availability computation (IMS fetch + conflict check) shall complete within acceptable time for interactive use. ● **Technical Constraint:** Room metadata must originate from IMS, not manually maintained.
Pre condition:	User must be logged into the portal.

Flow:	Main (success) Flow: 1. User selects desired date and time range. 2. System fetches room list from IMS. 3. System queries bookings table for conflicts in the selected slot. 4. System displays rooms as green (available) or red (unavailable) squares
	Alternate Flows: 2a. IMS unavailable → Show cached data with warning “Live data not available”.
Post Condition:	The user successfully views the current availability grid for rooms.

Use Case	UC-03
Use Case Name	Search Rooms
Overview:	**A user applies filters like room type (classroom/meeting), capacity, or date to narrow down available rooms. The system queries the database and updates the display in real-time. This ensures efficient selection without scrolling through all rooms.**
Actors:	Primary: User (Staff / Faculty)
Properties:	**Scalability:** Search functionality shall continue to perform acceptably as the number of rooms and bookings increases.
Pre condition:	User is on the room availability/search page.
Flow:	Main (success) Flow: 1. User selects filter options (type: classroom/meeting, capacity range, date/time). 2. System re-queries database with filters. 3. Grid refreshes instantly showing only matching rooms.
	Alternate Flows: 1a. No rooms match filters → Display “No rooms found” message.
Post Condition:	Grid displays only rooms that satisfy the applied filters.

Use Case	UC-04

Use Case Name	Submit Booking Request
Overview:	**A user selects a room, specifies start/end time, purpose, and submits the request to CFS. The system saves it as 'pending' in the database and sends a notification email to CFS. This initiates the approval workflow.**
Actors:	Primary: User (Staff / Faculty).
Properties:	**Reliability:** Booking submission shall be atomic—either fully stored as *pending* or not stored at all.
Pre condition:	User has identified an available room for their desired time slot.
Flow:	Main (success) Flow: 1. User fills purpose and optional snacks/tech. 2. System checks final conflict. 3. Booking saved as 'pending' in database. 4. Email sent to CFS. 5. Success message shown to user.
	Alternate Flows: **2a. Time Conflict Detected** - At step 2, if the system detects that selected room is not available for the given time slot) - The system displays an error message indicating the conflict and the booking request is not saved. - The user is prompted to choose another room or another slot for the same room.
Post Condition:	Booking record exists with status 'pending' and CFS has been notified.

Use Case	UC-05
Use Case Name	View Pending Requests
Overview:	**An admin views a list of all pending booking requests with details like user, room, time, and purpose. The system fetches from the database and sorts by submission time. This enables efficient review and decision-making.**
Actors:	Primary: Admin (CFS)
Properties:	**Process Constraint:** Pending requests shall be ordered by submission time to ensure fair processing.
Pre condition:	Admin must be logged into their dashboard.
Flow:	Main (success) Flow: 1. Admin opens "Pending Requests". 2. System fetches all 'pending' bookings. 3. Displays table with user, room, time, purpose.

Post Condition:	Admin sees current list of pending requests.

Use Case	UC-06
Use Case Name	Approve booking
Overview:	**An admin reviews a pending request and approves it, updating the status in the database. The system sends a confirmation email to the user and marks the room as unavailable for that slot. This finalizes the booking for use.**
Actors:	Primary: Admin (CFS)
Properties:	**Consistency:** Approval shall immediately block the room for the approved time slot across the system. **Concurrency:** The system shall prevent race conditions where multiple admins attempt to approve conflicting requests.
Pre condition:	Admin is viewing a specific pending booking request.
Flow:	Main (success) Flow: 1. Admin clicks “Approve”. 2. System updates status to ‘approved’. 3. Emails confirmation to user. 4. If snacks selected → email to CDS.
	Alternate Flows: 2a. Time conflict now exists → Prevent approval, show error.
Post Condition:	Booking status is ‘approved’, user and (if applicable) CDS are notified.

Use Case	UC-07
Use Case Name	Decline booking
Overview:	**An admin rejects a pending request, adding notes for the reason, and updates the status. The system notifies the user via email with the decline details.**
Actors:	Primary: Admin (CFS)

Properties:	● **Auditability:** Decline actions shall store the rejection reason for future reference.
Pre condition:	Admin is viewing a specific pending booking request.
Flow:	Main (success) Flow: 1. Admin selects “Decline”, enters reason. 2. Status updated to ‘declined’. 3. Email with or without reason is sent to user.
	Alternate Flows:
Post Condition:	Booking marked ‘declined’ and user notified via mail.

Use Case	UC-08
Use Case Name	Logout
Overview:	**A user or admin ends their session securely. The system clears the session data and redirects to the login page. This prevents unauthorized access after use.**
Actors:	Primary: User (Staff / Faculty), Admin (CFS)
Properties:	● **Security:** Session tokens must be completely invalidated on both the client and server side immediately upon execution to prevent session hijacking. ● **Performance:** The logout process and redirection to the CAS login screen must complete in under 1 second.
Pre condition:	The user/admin is currently logged into an active session.
Flow:	Main (success) Flow: 1. User clicks “Logout”. 2. System destroys session. 3. Redirects to login page.
	Alternate Flows:
Post Condition:	Session is terminated and user is on login page.

Use Case	UC-09
Use Case Name	Cancel Booking
Overview:	**A user cancels an existing pending or approved booking before the start time. The system updates the status to 'cancelled', notifies CFS via email, and releases the room availability. This provides flexibility for changes in plans.**
Actors:	Primary: User (Staff / Faculty)
Properties:	● **Performance:** The system must update the database and release the room's availability in real-time (under 2 seconds) to prevent subsequent booking errors. ● **Reliability:** The automated email notification to CFS must include a retry mechanism (e.g., up to 3 attempts) in case the mail server experiences temporary downtime.
Pre condition:	User has an active booking (pending or approved) that hasn't started yet.
Flow:	Main (success) Flow: 1. User selects booking from history. 2. Clicks “Cancel”. 3. Status changed to ‘cancelled’. 4. Email sent to CFS..
	Alternate Flows:
Post Condition:	Booking is cancelled, and room slot is freed.

Use Case	UC-10
Use Case Name	Snack Options (CDS integration)
Overview:	**During booking submission, a user selects optional snacks. The system records these in the booking details for approval. If approved, it triggers notifications to CDS.**
Actors:	Primary: User (Staff / Faculty) Supporting: CDS (Campus Dining Services)
Properties:	● **Interoperability:** The system must reliably and securely transmit the approved snack data to the Campus Dining Services (CDS) system using standard, lightweight data formats (like JSON over a REST API). ● **Usability:** The UI for selecting optional items must clearly display any limits or constraints so users do not submit invalid requests.

Pre condition:	User is in the process of submitting a booking request.
Flow:	Main (success) Flow: 1. User checks snacks/tech options. 2. Details saved with booking record. 3. On approval, CDS is notified .
	Alternate Flows:
Post Condition:	Booking record includes selected snacks options.

Use Case	UC-11
Use Case Name	Submit Feedback
Overview:	**After an event ends, a user receives a dropdown box for submitting feedback. The system stores it in the database and marks the booking as completed. This helps improve future services based on user input.**
Actors:	Primary: User (Staff / Faculty)
Properties:	• **Usability:** The feedback form must be fully responsive and easily navigable as users are likely to submit it on the go after an event.
Pre condition:	After any particular event such as booking is approved that may require feedback for improvement.
Flow:	Main (success) Flow: 1. System (via cron job) detects that event has ended and updates booking status to “completed”. 2. System sends feedback email with link to the user. 3. User clicks the link and optionally fills rating (1–5) and comments. 4. If user submits, feedback is saved in database.
	Alternate Flows: 3a. User ignores the email or does not submit the feedback. → No feedback record is created (booking remains “completed”).
Post Condition:	Booking status is “completed” (regardless of whether feedback was submitted or not). If the user submitted feedback, it is stored in the feedback table linked to the booking.

Use Case	UC-12
Use Case Name	Request More Details

Overview:	**An admin requests additional information for a pending booking by adding notes. The system sends an email to the user prompting for updates and keeps the status as 'pending'. This allows clarification without outright rejection.**
Actors:	Primary: User (Staff / Faculty), Admin (CFS)
Properties:	● **Data Integrity:** The system must ensure that the original booking request data remains entirely intact and locked from other edits while its status is pending clarification from the user. ● **Performance:** The system must dispatch the clarification email to the user within 1 minute of the admin requesting clarification.
Pre condition:	Admin is reviewing a pending booking request that lacks necessary information.
Flow:	Main (success) Flow: 1. Admin adds notes and clicks “Request Details”. 2. Status remains ‘pending’. 3. Email sent to user with notes.
	Alternate Flows:
Post Condition:	User receives request for more details; pending status remains unchanged.

Use Case	UC-13
Use Case Name	View statistics/calendar
Overview:	**An admin accesses charts and a calendar view of bookings, filtered by time or status. The system uses libraries like Chart.js and FullCalendar.js to visualize data from the database. This provides insights for resource management.**
Actors:	Primary: Admin (CFS)
Properties:	● **Performance:** The visual charts and calendars (using Chart.js and FullCalendar.js) must render within 3 seconds, even when querying large datasets spanning an entire academic year. ● **Scalability:** Database queries generating these statistics must be properly indexed to ensure the admin dashboard does not suffer performance degradation as the booking history grows over time.
Pre condition:	Admin is logged in and navigates to the analytics dashboard.
Flow:	Main (success) Flow: 1.Admin navigates to the “Analytics” section from the Admin Dashboard.. 2.System queries the database for records related to approved booking request and pending requests. 3.The system displays:

	A calendar view showing room bookings mapped to dates and time slots. **Statistical charts** (e.g., total bookings, bookings per room, approval rates, monthly usage). 4.Admin optionally applies filters (date range, room, status). 5.The system refreshes the calendar and charts based on selected filters.
	2a. Alternate flow: No booking data available → System displays empty calendar and a message such as “No data available”.
Post Condition:	Admin gains a visual overview of room usage, booking trends, and scheduling patterns. (Note: This is a read-only operation)

Use Case	UC-14
Use Case Name	View booking history
Overview:	**A user accesses a list of their past and pending bookings, including details like room, time, and status. The system retrieves data from the bookings table and displays it in a table format. This allows users to track their requests easily.**
Actors:	Primary: User (Staff / Faculty)
Properties:	● **Performance:** The history table should utilize some level of optimization so that users with extensive booking histories experience page load times of under 2 seconds. ● **Usability:** The tabular display must include immediate client-side sorting (e.g., by date or status) without requiring a full page refresh.
Pre condition:	User is logged in to the portal.
Flow:	Main (success) Flow: 1. User navigates to their "Booking History" tab. 2. System queries the database for records tied to the user. 3. System displays a tabulated list of past and pending requests
	Alternate Flows: 2a. If the system has never made a booking, the system displays an message saying no booking done yet.
Post Condition:	User sees their complete history of bookings.